# scripts/ ownership_precondition.sh â€” Ownership Startup Precondition

**Revision:** 1 **Last modified:** 2026-08-21T14:58:00Z **Purpose:** Operator guide for the startup check that refuses to start the system when a declared location cannot produce operator-owned files. **Last verified:** 2026-08-21

---

## Overview

`scripts/ownership_precondition.sh` answers exactly one question, for every location listed in `config/owned_paths.yaml`:

> If this system writes a file here, will that file belong to the person who started the system?

If the answer is â€œnoâ€ anywhere, the script exits non-zero so startup can be refused. This is **fail-closed by operator decision** (clarify session, 2026-08-21): starting with a warning was explicitly rejected, because a missed warning silently reproduces the defect the feature exists to remove â€” content landing at an identity (uid `100999` under a rootless container mapping) that has no host account, so the operator could not read, move, or delete their own downloads, and could not read their own credential database.

### Why it PROBES instead of inspecting

**Ownership of a location cannot be read off the location itself.** This is the single most important thing to understand about this script, and it is measured fact rather than caution: the download root was owned by the operatorâ€™s uid **and still received files at uid 100999**. That is the defect. So the directoryâ€™s own owner, the configured `PUID`, and â€œno error occurredâ€ are all *proxies* for the condition â€” and Â§11.4.201 forbids asserting a proxy in place of the condition it stands for. A probe that passes because it never wrote anything is a false pass.

Every verdict this script emits therefore comes from creating a **real file** in the **real location** and reading the owner back from the host, then removing the file.

### Why there are TWO probes, and why neither speaks for the other

| Probe | What it does | What it proves |
|---|---|---|
| **P2** (host write) | | |

| Probe | What it does | What it proves |
|---|---|---|
| | The script, running as the operator, creates a file in the location, reads its owner, removes it | The location exists, is writable, and is on a filesystem that carries ownership at all. **Necessary, never sufficient.** |
| **P1** (container write) | A throwaway container writes into the location as the *serviceâ€™s declared identity*, then the owner is read back **from the host** | The real condition. This is the only probe that can observe the reported defect. |

P2 alone is not enough for a reason that was measured during Phase 1 of this feature: a host-side write by the operator produces an operator-owned file **even in the directory whose container writes land at 100999** â€” `probe_location` returned `ok` for exactly that directory. So the report keeps the two probes separate and never dresses a P2 pass up as a P1 pass.

Declared **files** (the credential store, `config/boba.db`) are the one carve-out: there is nothing to create beside them, and reading the targetâ€™s own owner IS the real condition there rather than a proxy for it. Files get no P1 probe.

# Prerequisites

- `bash` 4+, `stat`, `mktemp` (coreutils)
- `python3` with **PyYAML** â€” used to parse `config/owned_paths.yaml` and `docker-compose.yml`. The interpreter is *probed*, not assumed: a python3 that cannot `import yaml` is treated as unusable, and the script exits 2 rather than producing a confusing downstream failure.
- `scripts/lib/ownership.sh` â€” sourced, never re-implemented (scope resolution, operator uid, and the probe itself live there and are shared with `scripts/ownership_repair.sh` and the pre-build gate).
- `config/owned_paths.yaml` â€” the declared scope (data-model **E1**).
- **Optional:** `podman` or `docker`. Without a container runtime the P1 probe cannot run; the script reports an honest SKIP rather than refusing, see *Edge cases*.

# Usage examples

## Example 1 â€” the normal invocation

```
scripts/ownership_precondition.sh
```

Prints a per-location verdict, then one of `OWNERSHIP-PRECONDITION: OK`, `... FAIL`, or `... CANNOT-RUN`.

### Example 2 â€” quiet mode, for a startup path

```
scripts/ownership_precondition.sh --quiet
```

Suppresses passing detail and the success banner. It does **not** suppress a refusal, a cannot-run report, or a probe-coverage gap â€” see *Flags*.

### Example 3 â€” check a different scope file

```
scripts/ownership_precondition.sh --scope /path/to/other_owned_paths.yaml
scripts/ownership_precondition.sh --scope=/path/to/other_owned_paths.yaml
```

Both forms are accepted. `--scope` takes precedence over the `OWNED_PATHS_FILE` environment variable.

### Example 4 â€” declare that there is no container runtime

```
CONTAINER_RUNTIME= scripts/ownership_precondition.sh
```

An explicitly **empty** `CONTAINER_RUNTIME` means â€œthere is no runtimeâ€ and is honoured as such â€” P1 is skipped and no container is launched. This is how test suites and gates run the script without spawning containers. Contrast with the variable being **unset**, which means â€œnot decided yetâ€ and makes the script detect `podman`/`docker` itself.

### Example 5 â€” use in a shell conditional

```
if scripts/ownership_precondition.sh --quiet; then
    echo "safe to start"
else
    rc=$?
    echo "refused (exit ${rc}) â€” see the report above"
fi
```

Note that 1 and 2 mean different things and must not be collapsed; see *Exit codes*.

## Flags

| Flag | Effect |
|---|---|
| `--scope <path>` / `--scope=<path>` | Scope file to check. Overrides `OWNED_PATHS_FILE`. A missing argument to the space-separated form is exit 2, not a silent default. |
| `--quiet` | Suppress per-location OK detail and the `OWNERSHIP-PRECONDITION: OK` banner. Refusals, cannot-run reports, and probe-coverage gaps are **always** printed. |
| `-h`, `--help` | Print usage and exit `0`. |

| Flag | Effect |
| --- | --- |
| *(anything else)* | Unrecognised: usage to stderr, then exit 2. An unknown flag means the caller asked for something this script does not do; ignoring it would silently run a *different* check than the one requested. |

**Why `--quiet` hides only good news.** A quiet flag that could hide a refusal â€” or hide the fact that the probe which observes the real defect never ran â€” would re-create the missed-warning failure the fail-closed decision was made to avoid. The implementation splits output into `say()` (silenceable) and `say_always()` (never silenced), and every failure path uses `say_always`.

# Env vars

| Variable | Meaning |
| --- | --- |
| `OWNED_PATHS_FILE` | Scope file path when `--scope` is absent. Read by `scripts/lib/ownership.sh`; default `config/owned_paths.yaml` under the project root. |
| `CONTAINER_RUNTIME` | **Set-but-empty** = no runtime available (P1 skipped, nothing detected). **Unset** = detect `podman` then `docker`. **Non-empty** = use that command verbatim. Absent and empty are deliberately different states â€” this mirrors `start.sh`, which sets the empty value precisely to mean â€œnone foundâ€. |
| `PYTHON_BIN` | Consulted first by `ownership_python()` when choosing an interpreter (then `.venv/bin/python`, then `python3`). Each candidate must actually `import yaml` to be selected. |
| *scope placeholders* | Any `${VAR:-default}` inside a declared path is expanded against the live environment â€” `QBITTORRENT_DATA_DIR` is the one that matters in the shipped scope, so the host-specific download root resolves at run time instead of being hardcoded (Â§11.4.35). |

No credential value is ever read, printed, or logged by this script.

# Exit codes

| Code | Meaning |
|---|---|
| `0` | Every declared location can produce operator-owned files. |
| `1` | At least one cannot â€” **startup MUST NOT proceed.** Each offending location is named individually. |
| `2` | **The check could not run at all.** |

**2 is not a pass, and it is not a failure of the system either.** It means the check asserted *nothing*: the scope file was missing, unparseable, or empty; no python3 with PyYAML was available; or the caller passed an unusable argument. Reporting â€œcould not runâ€ as success is the Â§11.4.201(6) blind-instrument failure â€” a blind instrument and a clean system return the same quiet zero â€” so this script refuses to conflate them. The `CANNOT-RUN` report always names the cause and ends with the literal line:

```
This check asserted NOTHING. It is not a pass â€” fix the cause above and
```

A caller that treats 2 as success has re-introduced the exact class of defect this feature exists to remove.

# Edge cases

- **A declared path that is absent and `optional: true`** â†’ not a failure. Reported as `absent, declared optional â€” not a failure` and skipped. `config/boba.db` is exactly this shape before first boot, and this case is the contractâ€™s mandated **negative control**: a guard that refuses a healthy system is as broken as one that passes a broken one (Â§11.4.201(1)).
- **A declared path that is absent and not optional** â†’ failure (exit 1). Per data-model E1 an absent non-optional path is an error, not a skip.
- **No container runtime** â†’ P1 is skipped with the named reason `runtime_unavailable`, and the script says out loud that nothing here asserts a service write would land at the operatorâ€™s uid. It does **not** refuse: refusing every host without a runtime would be a Â§11.4.201(1) false-positive refusal, which is exactly as forbidden as a false pass.
- **P1 could not run for another reason** â†’ also a named SKIP, never a refusal. The reasons are a closed set: `runtime_unavailable`, `route_unavailable` (compose file unreadable), `route_undeclared` (no compose service mounts this location), `service_image_unavailable` (no locally-present image, so reproducing the service would need a network pull at startup), `probe_container_failed(<service>: <first 120 chars of stderr>)`, and `probe_file_not_found(<service>)` (the container reported success but nothing landed on the host â€” the probe observed nothing, so it asserts nothing).

- **`docker-compose.yml` missing or unparseable** â†’ routes are reported as `route information unavailable`, not as â€œno route declaredâ€�. An unread file and a file declaring nothing are different findings (Â§11.4.201(6)).
- **A compose service with no declared ownership route** â†’ reported as `NO route declared (neither userns_mode: keep-id nor PUID=0)`, but this alone does **not** refuse. A route reading is *configuration*, and the contract forbids inferring ownership from configuration; turning a config reading into a refusal would make this check refuse on a proxy, inside the check written to forbid proxies. Route **completeness** is enforced where it belongs: the pre-build gate `scripts/pre_build/check_cm_ownership_invariants.sh` (invariant 33, `CM-OWNERSHIP-INVARIANTS`).
- **An unrecognised verdict from either probe** is a failure, not a pass â€” â€œunrecognised is not cleanâ€�.
- **A probe file left behind**: both probes remove their probe file, including on the container-failure path. The host probe uses `mktemp` inside the probed directory; the container probe uses a `$$/$RANDOM`-suffixed marker name.
- **Wiring, measured 2026-08-21T15:10Z:** `start.sh` calls this script from `run_ownership_gate()`, which runs on the normal start path **after** the directory-creation stages and **before** any container writes (also on the `--recreate` path). The ordering is deliberate: the declared download root and `config/` are `optional: false`, so probing them before they exist would refuse a healthy fresh checkout â€” the Â§11.4.201(1) false-positive refusal. `start.sh` treats **both** exit `1` and exit `2` as refuse-to-start, and a missing script file as refuse-to-start; only exit `0` proceeds. It runs the script under `nice -n 19 ionice -c 3` when both tools are present. The systemd unit `scripts/systemd/user/boba-stack.service` deliberately does **not** declare its own `ExecStartPre` for this: it invokes `./start.sh --no-build` and inherits the single gate, so the two paths cannot contradict each other about whether the precondition ran. This wiring landed while this document was being written â€” it was measured directly in the working tree, not taken from the contract (Â§11.4.6).

# Internal behaviour

1. **Parse arguments.** `--scope` is exported as `OWNED_PATHS_FILE` so the shared library sees it (the libraryâ€™s documented override, not a backdoor).
2. **Read the scope.** `ownership_scope_entries` emits one TAB-separated row per declared entry: `<path>\t<kind>\t<optional>\t<preserve_mode>\t<recursive>`. A read failure is exit 2; a scope with zero locations is also exit 2 â€” â€œa precondition that checked zero locations has verified nothingâ€�.
3. **Resolve the runtime** (see `CONTAINER_RUNTIME` above) and **read the compose routes** â€” one row per service carrying its image, `userns_mode`, `PUID`, and normalised mount sources, with `${VAR:-default}` expanded *before* the `host:container` split (splitting first would tear a default value containing a colon in half).

4. **Per declared location:**
    1. Run **P2** via `probe_location`. Verdicts are `ok`, `absent`, `unwritable`, `wrong-owner:<uid>` (data-model **E4**).
    2. If the location is a regular **file**, stop here â€” its own owner is the real condition.
    3. Otherwise run **P1**: pick a compose service whose mount contains or is contained by this location *and* whose image is already present locally, then run `<runtime> run --rm --network=none [--userns=<mode>] [--user <PUID>] --entrypoint /bin/sh -v <dir>:/ownership-probe:z <image> -c "touch â€¦"` under a 120-second `timeout`, `stat` the resulting file **on the host**, and remove it. The imageâ€™s own entrypoint is bypassed on purpose: the probe must perform a bare write, not boot the service.
5. **Print the probe-coverage report** whenever any P1 was skipped â€” always, `--quiet` included. It names each skipped location and reason, states that nothing there asserts a service write would land at the operatorâ€™s uid, and prints the declared route as the clearly-labelled configuration-only fallback.
6. **Verdict.** Any failure â†’ `OWNERSHIP-PRECONDITION: FAIL`, every offending location named, plus the actionable line `Startup refused. Fix the location(s) above, or run scripts/ownership_repair.sh.` Otherwise `OWNERSHIP-PRECONDITION: OK`.

### Two implementation details worth knowing

- **Path containment is checked in both directions.** A service that mounts an *ancestor* of a declared location writes into it, and a service that mounts a *subdirectory* of it also writes into it. Matching one direction only would silently drop half the services that can produce files there.
- **TAB-separated rows are split with `readarray -d $'\t'`, not `IFS=$'\t' read`.** TAB is an IFS *whitespace* character, so `read` collapses a run of tabs into one delimiter â€” a row with empty middle fields (a compose service built from a Dockerfile has no `image:`) then shifts every later field left and the mounts column reads as EMPTY. Measured against this repositoryâ€™s own `docker-compose.yml` on 2026-08-21: the first version of the script used the collapsing read and reported â€œno compose service mounts this locationâ€ for `config/` and for the download root **while five services mount them** â€” a false statement produced by the instrument rather than by the system (Â§11.4.201(7)(c), â€œthe path is part of the instrumentâ€). The unit fixture scope has no compose service at all, so the suite could not see this; only running the real invocation against the real scope did.

## Related scripts

- ownership_repair.md â€” `scripts/ownership_repair.sh`, the remediation this scriptâ€™s failure message points the operator at. The precondition **detects**; the repair **fixes**.
- `scripts/lib/ownership.sh` â€” the shared library both scripts source (`ownership_scope_entries`, `ownership_operator_uid`,

`probe_location`, `ownership_scope_fingerprint`). Sourced rather than forked: a second copy would be the near-identical fork Â§11.4.251 forbids and would drift from the repair and the gate that read the same scope.
- `scripts/pre_build/check_cm_ownership_invariants.sh` â€” invariant 33 (`CM-OWNERSHIP-INVARIANTS`) of `scripts/pre_build_verification.sh`. Enforces route **completeness** across compose services, which this script deliberately only reports.
- `config/owned_paths.yaml` â€” the declared scope (E1). Adding an entry here changes the scope fingerprint, which re-arms the repair.
- `tests/unit/test_ownership_precondition.sh` â€” the paired contract suite.
- `tests/ownership/test_container_writes_owned_files.py` â€” the Â§11.4.115 RED that captured the original defect at uid 100999.
- `../guides/file-ownership.md` â€” the operator-facing narrative for the whole ownership feature.

# Cross-references

- `specs/002-user-owned-downloads/contracts/startup-precondition.md` â€” the contract this script implements (FR-010, FR-010a, FR-010b), including the 2026-08-21 amendment (finding X1) that split the single probe into P1/P2.
- `specs/002-user-owned-downloads/data-model.md` â€” **E1** scope, **E4** probe verdicts, **E5** service ownership routes.
- **Â§11.4.201** â€” every guard asserts the real condition; a false-positive refusal is a FAIL-bluff exactly as forbidden as a false-negative pass. This is the anchor behind the probe, behind the P1-skip-not-refuse rule, and behind exit 2 being distinct from exit `0`.
- **Â§11.4.6** â€” no guessing: an unreadable scope is reported as unreadable, not as empty.
- **Â§11.4.3** â€” honest SKIP-with-reason, which is what an unavailable P1 emits.

# Anti-bluff & Â§11.4 discipline

- Every verdict comes from a **file that was actually created and whose owner was actually read back**. No verdict is derived from a directoryâ€™s own owner, from `PUID`, or from the absence of an error.
- The probe that can observe the real defect (P1) is never *implied* by the one that cannot (P2). When P1 does not run, the report says so, unconditionally, and labels the configuration fallback as configuration.
- `2` (cannot run) is structurally separate from `0` (pass), so a blind instrument can never be read as a clean system.
- `--quiet` cannot hide a refusal or a coverage gap.
- Â§11.4.263: this script signals no processes â€” there is no `kill`, `pkill`, or `killpg` anywhere in it, so the `pgid <= 1` broadcast-kill hazard cannot arise.

# Last verified

2026-08-21 â€” every flag, exit code, environment variable, and skip reason in this document was read from `scripts/ownership_precondition.sh` and `scripts/lib/ownership.sh` at commit-time state, not from the scriptâ€™s own usage text alone. The `start.sh` wiring is described from the invocation actually present in the working tree at 2026-08-21T15:10Z (`run_ownership_gate()`, called at two sites), not from the contractâ€™s word for it.